

SocioUnido: Ecosistema SaaS inteligente para la gestión descentralizada y control de accesos offline en instituciones deportivas

Felipe Santino Ascencio, Lautaro Gabriel Ghosn, Martín Guerrero, Axel Zielonka

Facultad de Ingeniería

Universidad de Buenos Aires (FIUBA)

Buenos Aires, Argentina

{fascencio, lghosn, mguerrero, azielonka}@fi.uba.ar

Resumen—El presente artículo documenta el diseño, desarrollo e implementación de SocioUnido, una plataforma integral *Software as a Service* (SaaS) bajo un modelo E2E (*Enterprise-to-Enterprise*) orientada a modernizar la gestión administrativa y el control de accesos en clubes deportivos argentinos. La solución aborda de manera directa tres ineficiencias críticas del sector: la morosidad societaria silenciosa, el fraude en accesos por préstamo de credenciales y la subutilización de activos institucionales. Tecnológicamente, la plataforma propone un ecosistema modular de vanguardia que incluye: un control de accesos perimetral descentralizado basado en algoritmos TOTP (*Time-Based One-Time Password*) capaz de operar 100 % *offline* para superar el colapso sistemático de redes en estadios; un asistente conversacional cognitivo (BotIn) soportado por procesamiento de lenguaje natural (PLN) para la autogestión sin fricción; y un motor analítico para la prevención proactiva del abandono (*churn*). Validado mediante metodologías ágiles, auditorías DevSecOps y testing automatizado con agentes de IA, SocioUnido demuestra ser una herramienta robusta y escalable que promueve el saneamiento financiero institucional y la inclusión tecnológica.

Index Terms—SaaS, Control de Accesos Offline, TOTP, Machine Learning, DevSecOps, Gestión Deportiva, QA Automatizado.

I. INTRODUCCIÓN

I-A. Contexto y motivación

Las asociaciones civiles deportivas sin fines de lucro en la República Argentina constituyen pilares fundamentales para la cohesión social, la contención comunitaria y la promoción de actividades culturales y recreativas. A diferencia del modelo societario de propiedad privada imperante en otras latitudes, el club deportivo argentino opera tradicionalmente bajo un esquema democrático donde el socio es el verdadero propietario y gestor de la institución. No obstante, a pesar de su incalculable valor sociocultural, la inmensa mayoría de estas entidades de tamaño mediano y pequeño —desde los clubes de las categorías de ascenso del área metropolitana de Buenos Aires hasta las ligas regionales del interior del país— enfrentan un escenario de marcada vulnerabilidad financiera, profundizado por la subsistencia de prácticas de gestión administrativa marcadamente tradicionales y reactivas.

Históricamente, la adopción de herramientas tecnológicas avanzadas ha sido un privilegio exclusivo de las escasas

corporaciones deportivas que de forma sistemática acceden a los recursos de la élite profesional de primera división. Para las instituciones del ascenso y del interior, la digitalización de sus operaciones suele limitarse a la implementación de planillas de cálculo manuales aisladas, sistemas de base de datos desactualizados o simples sistemas de registro de transacciones genéricos de tipo reactivo, conocidos técnicamente como sistemas *CRUD* básicos (crear, leer, actualizar y borrar). Esta brecha tecnológica genera un aislamiento operativo que restringe la capacidad de las dirigencias para automatizar procesos administrativos rutinarios, optimizar la recaudación de cuotas, prevenir fugas de capital y garantizar la seguridad perimetral de sus instalaciones durante eventos de concurrencia masiva.

Frente a esta realidad, el presente trabajo documenta el diseño, desarrollo e implementación de **SocioUnido**, un ecosistema de *software* modular concebido bajo un modelo de *software* como servicio (*SaaS*) y marca blanca, específicamente dimensionado para resolver las demandas operativas reales de las instituciones deportivas argentinas. El proyecto no se concibe como una simple herramienta contable digital, sino como una plataforma inteligente, descentralizada y proactiva de inteligencia de negocios y ciberseguridad aplicada al saneamiento financiero y la inclusión tecnológica institucional.

I-B. Planteamiento del problema y faltante en el sector

El análisis de dominio realizado sobre la gestión operativa de los clubes deportivos en Argentina permitió identificar tres ineficiencias críticas de carácter silencioso que impactan directamente sobre su rentabilidad económica y su sostenibilidad institucional:

1. **La morosidad societaria silenciosa:** Debido a la desconexión existente entre las actividades deportivas de los socios, los registros administrativos y los medios de pago disponibles, las instituciones deportivas sufren una pérdida constante de ingresos por cuotas sociales impagas. La falta de canales digitales inmediatos y la dependencia de cobros manuales presenciales en tesorerías centralizadas fomentan un comportamiento de olvido por parte de los usuarios. Al no contar con alertas

proactivas ni motores capaces de anticipar la morosidad o la intención de baja societaria, los clubes actúan reactivamente una vez que la deuda se ha consolidado y la pérdida del socio es irreversible.

2. **El fraude en el control de accesos por duplicación o préstamo de credenciales físicas:** En eventos masivos, la utilización de credenciales impresas plásticas con códigos de barras estáticos o sistemas magnéticos heredados propicia el préstamo de carnets entre socios activos y personas no autorizadas. Esta duplicidad de accesos genera no solo una severa fuga de recaudación por entradas no vendidas, sino también una grave vulnerabilidad de seguridad lógica y física en los puntos de control del estadio. La solución tradicional exige conectividad permanente de banda ancha de alta velocidad en los molinetes perimetrales para validar la validez de los códigos contra la base de datos central en la nube. Sin embargo, en escenarios masivos, la saturación por radiofrecuencia y la caída recurrente de las redes de telefonía celular móvil imposibilitan la validación en tiempo real (*online*), obligando en muchas ocasiones al personal de control a liberar los ingresos para evitar desmanes físicos, validando involuntariamente accesos fraudulentos o de socios morosos.
3. **La subutilización de instalaciones comunes, disciplinas y activos deportivos rentables:** Gran parte de los clubes poseen instalaciones ociosas (como canchas auxiliares, quinchos, salones o predios de entrenamiento) e infraestructura de disciplinas deportivas infrautilizadas debido a la ausencia de un canal centralizado de reservas. Los socios deben acudir presencialmente o comunicarse telefónicamente en horarios acotados para consultar disponibilidad, lo que desincentiva el consumo interno y reduce las fuentes de financiamiento alternativas de la institución.

SocioUnido nace con el firme propósito de mitigar estas problemáticas de raíz, integrando en una única plataforma la gestión administrativa y social proactiva, la prevención analítica de la morosidad y un sistema de control de accesos seguro y descentralizado capaz de validar ingresos masivos de forma 100 % fuera de línea (*offline*).

I-C. Objetivos del Trabajo Profesional

I-C1. Objetivo general: Diseñar, desarrollar y validar un ecosistema de *software* integral y adaptativo de marca blanca que centralice la administración de clubes deportivos, optimice la retención y fidelización societaria mediante mecanismos cognitivos y predictivos, y blinde el perímetro de las instalaciones mediante una solución criptográfica de control de accesos descentralizada y tolerante a fallos de conectividad.

I-C2. Objetivos específicos: Para alcanzar el objetivo general, se definen las siguientes metas técnicas y metodológicas:

- Implementar una arquitectura perimetral y distribuida de microservicios resiliente, escalable y cibersegura, orquestada mediante contenedores y centralizada bajo un portal de enlace de aplicaciones (*API gateway*) unificado.

- Desarrollar una aplicación web progresiva (*PWA*) adaptativa para el socio, compatible con dispositivos móviles de bajos recursos, que centralice su credencial digital, la reserva de espacios, la inscripción a disciplinas y el flujo transaccional de pagos.
- Diseñar y validar un algoritmo de control de accesos inteligente basado en códigos criptográficos de un solo uso basados en tiempo (*TOTP*), que permita la validación instantánea del estado de pago e identidad del socio sin requerir conectividad de red (*offline*) en los dispositivos de escaneo del club.
- Construir un asistente transaccional conversacional interactivo cognitivo (asistente “BotIn”), con procesamiento de lenguaje natural (*PLN*), para reducir la fricción transaccional administrativa a cero.
- Diseñar y desarrollar una plataforma web de administración centralizada que permita a las autoridades del club digitalizar de forma integral los procesos institucionales, automatizar la facturación y cobranza de cuotas sociales, y gestionar de manera eficiente las disciplinas deportivas y los activos compartidos.
- Validar metodológicamente el proceso de ingeniería de *software* utilizando el marco ágil Scrum y certificar la calidad, ciberseguridad e interoperabilidad del código fuente de forma continua mediante el análisis estático y un agente autónomo de aseguramiento de calidad (*QA*) basado en inteligencia artificial.

I-D. Articulación e integración de conocimientos académicos de la carrera

El diseño y materialización del ecosistema SocioUnido no constituye una actividad de desarrollo tecnológico aislada, sino la síntesis práctica y la convergencia multidisciplinar de los conocimientos teóricos adquiridos a lo largo de toda la trayectoria académica en la Facultad de Ingeniería de la Universidad de Buenos Aires. La resolución de los desafíos del dominio deportivo y de infraestructura requirió articular de forma sinérgica conocimientos de áreas diversas de la informática y de la formación general del ingeniero:

- **Ciencias básicas e infraestructura física:** Los fundamentos de electromagnetismo, atenuación de ondas de radiofrecuencia y saturación de espectro analizados en las asignaturas de Física y Física para Informática proveyeron el sustento científico para predecir el colapso de las antenas celulares en los estadios de fútbol ante altas densidades de usuarios. Esta base física justificó de manera irrefutable la necesidad de diseñar un sistema de control de ingresos descentralizado y desacoplado de la red en la nube.
- **Matemáticas avanzadas y criptografía:** Las teorías de aritmética modular, funciones unidireccionales y transformaciones matriciales estructuradas en Álgebra y Álgebra Lineal permitieron desarrollar y robustecer el algoritmo criptográfico del sistema de accesos *offline* mediante semillas *TOTP* e implementar firmas digitales con claves simétricas seguras e infalsificables.

- **Ingeniería de software, diseño y arquitectura distribuidos:** Los conceptos de encapsulamiento, modularidad y diseño orientado a objetos (entre muchos otros) provistos por Paradigmas de Programación y Taller de Programación guiaron la construcción de un código fuente limpio y mantenible. Por su parte, Base de Datos aportó el marco teórico del modelo relacional, control de transacciones concurrentes complejas y optimización de consultas SQL en PostgreSQL. Redes y Sistemas Distribuidos I fueron la base técnica para diseñar la topología de microservicios, la tolerancia a fallos y la comunicación interservicio asíncrona.
- **Ciberseguridad y defensa profunda:** La aplicación de los lineamientos de análisis de vulnerabilidades, sanitización de entradas, prevención de inyecciones y control estricto de roles de acceso mediante JSON Web Tokens (JWT) provienen directamente del Taller de Seguridad Informática, construyendo una infraestructura segura contra ataques perimetrales.
- **Emprendedurismo, ética y contexto profesional:** El desarrollo de un producto mínimo viable comercializable, el estudio macrocontextual bajo la metodología de las 5 C, el análisis microcontextual de las 4 P, el seguimiento de métricas *SaaS* y la gestión ágil *Scrum* se forjaron de manera directa sobre las bases conceptuales y procedimentales adquiridas a lo largo de un trayecto de cuatro materias clave: ingeniería del software I, ingeniería del software II, empresas de base tecnológica I y empresas de base tecnológica II. Estas asignaturas proveyeron el andamiaje metodológico para estructurar tanto el ciclo de vida del producto de software como su estrategia de viabilidad y comercialización en el mercado real.

II. ESTADO DEL ARTE

II-A. El paradigma tradicional de gestión deportiva y el *status quo*

El mercado de soluciones informáticas destinadas a la administración de instituciones deportivas en la República Argentina se ha caracterizado históricamente por un profundo conservadurismo metodológico y tecnológico. Las herramientas utilizadas por la inmensa mayoría de las comisiones directivas de clubes de mediana y pequeña envergadura evidencian un rezago estructural frente a las transformaciones digitales observadas en otras industrias de servicios masivos. El denominado *status quo* operativo se asienta sobre tres metodologías tradicionales que limitan severamente el control y el crecimiento institucional:

1. **Gestión analógica y descentralizada:** El uso de planillas impresas físicas en papel para el control diario de las instalaciones, actas manuales para el registro de los libros de socios y la conciliación contable presencial en ventanillas de tesorería centralizadas. Este enfoque no solo es propenso a errores humanos sistemáticos y pérdidas físicas de información, sino que demanda una asignación desproporcionada de horas-hombre en tareas administrativas rutinarias de escaso valor agregado.
2. **Soporte en herramientas ofimáticas genéricas:** El empleo de planillas de cálculo (como Microsoft Excel) operadas de forma aislada por los coordinadores de cada disciplina deportiva. La fragmentación de los datos impide consolidar un padrón único de socios, lo que genera inconsistencias críticas de información (como socios dados de baja que continúan asistiendo a actividades, o viceversa) y anula cualquier capacidad de análisis financiero global en tiempo real.
3. **Credenciales de identificación vulnerables:** La emisión de carnets de plástico físicos basados en tecnología de cloruro de polivinilo (PVC) con códigos de barras estáticos o bandas magnéticas heredadas. Estos soportes físicos presentan una alta susceptibilidad al desgaste material, la pérdida y, fundamentalmente, la falsificación o préstamo no autorizado. Esta práctica facilita el acceso de personas con cuotas sociales inactivas, consolidando una morosidad silenciosa y exponiendo la seguridad física y lógica del club durante eventos de alta concurrencia.

Las pocas instituciones que han implementado sistemas de software locales suelen depender de arquitecturas monolíticas locales instaladas en computadoras físicas dentro de las sedes. Estas soluciones exigen una inversión inicial significativa en infraestructura de servidores, sistemas de refrigeración y copias de seguridad manuales, además de someter al club a altos costos fijos de mantenimiento técnico y soporte especializado, lo cual resulta económicamente inviable para las finanzas de las asociaciones civiles de las categorías de ascenso.

II-B. Análisis crítico de los competidores actuales (caso de referencia: *Global.fan*)

Para evaluar la validez técnica y comercial de SocioUnido en el ámbito de la ingeniería de software aplicada, resulta indispensable contrastar su propuesta de valor frente a las plataformas de gestión deportiva que dominan el mercado del fútbol argentino profesional. El principal caso de referencia local es la plataforma *Global.fan*, considerada el competidor más representativo de la industria.

El análisis de arquitectura y alcance funcional de estos sistemas comerciales revela limitaciones estructurales críticas desde la perspectiva de la resiliencia tecnológica y el valor proactivo del software:

- **Naturaleza reactiva de la información (sistemas *CRUD* básicos):** Los competidores del mercado operan esencialmente como bases de datos estáticas basadas en un esquema de creación, lectura, actualización y borrado (*CRUD*). Estos sistemas dependen exclusivamente de la proactividad del usuario administrador o del socio para registrar un dato o gatillar una consulta. Carecen por completo de capas analíticas inteligentes que procesen los flujos de datos históricos para anticipar comportamientos, limitando la toma de decisiones a reportes históricos emitidos de manera posterior al hecho consumado (*ex post*).
- **Gestión de morosidad pasiva:** Ante la falta de pago de la cuota social, los sistemas vigentes se limitan a

generar un listado estático de morosos. La institución deportiva debe iniciar manualmente campañas de cobranza tardías sobre usuarios que ya se encuentran desvinculados emocional y financieramente del club. No existen mecanismos preventivos que detecten de forma temprana el riesgo latente de abandono (*churn rate*), cuando la probabilidad de recuperación del socio y del capital es estadísticamente superior.

- **Absoluta dependencia de conectividad perimetral:** Los sistemas dinámicos de acceso digital implementados por competidores comerciales utilizan códigos *QR* variables que se actualizan de forma constante en la aplicación del usuario. No obstante, para validar la legitimidad de dicho código en las puertas de acceso al estadio, el dispositivo lector o molinete perimetral debe realizar una petición de red (*online*) en tiempo real contra los servidores centrales de la plataforma en la nube. En escenarios de aglomeración masiva, el colapso sistemático de las celdas de telefonía móvil de cuarta y quinta generación (4G/5G) impide la comunicación de red, bloqueando los puntos de escaneo. Esto obliga a la dirigencia deportiva a liberar las puertas por motivos de seguridad civil, anulando la eficacia del sistema de accesos y posibilitando el ingreso de personas no autorizadas o morosas.
- **Aislamiento funcional e interfaces cerradas:** Los sistemas comerciales obligan al socio a descargar aplicaciones nativas pesadas, registrar nuevas cuentas, recordar contraseñas adicionales y navegar por plataformas ajenas a sus hábitos digitales diarios. Esta fricción en la experiencia de usuario incrementa la tasa de deserción en los flujos de autogestión y pago de cuotas. Asimismo, sus arquitecturas cerradas impiden la interoperabilidad con canales de mensajería masiva o asistentes conversacionales externos, centralizando la atención en oficinas físicas presenciales con horarios restringidos.
- **Enfoque comercial sesgado a la élite deportiva:** Las plataformas existentes basan su modelo de negocio en la captación agresiva de los clubes de primera división de gran envergadura institucional. La complejidad operativa de sus despliegues de hardware propietario en estadios y las altas tarifas de configuración inicial (*setup fees*) margina por completo a los clubes del ascenso metropolitano, el ascenso federal y las ligas regionales del interior, profundizando la brecha tecnológica y de seguridad en el fútbol local.

II-C. Ejes de innovación y disrupción tecnológica de SocioUnido

SocioUnido rompe disruptivamente con el estado de la técnica al introducir un ecosistema de ingeniería de software proactivo, descentralizado y cognitivo. La plataforma no opera como un mero registro pasivo del estado actual del club deportivo, sino que se anticipa a los eventos financieros y operativos mediante la aplicación de conceptos que redefinen los estándares de la industria local.

El carácter innovador y de vanguardia de la solución se consolida sobre cuatro ejes de alta ingeniería informática:

II-C1. 1. Control de accesos descentralizado y tolerante a fallos de red (*smart access offline*): La disrupción fundamental en materia de ciberseguridad perimetral es la eliminación absoluta de la dependencia de conectividad de red durante la validación de ingresos. El módulo de acceso inteligente de SocioUnido implementa un algoritmo criptográfico basado en contraseñas de un solo uso basadas en tiempo (*TOTP*, por sus siglas en inglés, *time-based one-time password*), estructurado de la siguiente forma:

- **Generación aislada en la interfaz del usuario:** La aplicación web progresiva (*PWA*) del socio genera un código *QR* dinámico que encripta de forma segura una semilla única de identidad y un sello de tiempo. Este proceso matemático se ejecuta localmente en el dispositivo celular del socio sin consumir datos móviles, permitiendo su correcto funcionamiento en escenarios de bloqueo total de red o incluso con el dispositivo en modo avión.
- **Validación perimetral desacoplada:** El personal de control, utilizando la aplicación operativa del club instalada en terminales móviles estándar, escanea el código *QR*.
- **Mitigación del fraude mediante protocolos *anti-replay*:** Para impedir la duplicación de accesos o el préstamo de carnets mediante capturas de pantalla, cada código *QR* posee una validez física estricta. Adicionalmente, el microservicio de acceso de SocioUnido interactúa con una caché distribuida en memoria (implementada mediante *Redis*) que almacena y rechaza de manera instantánea cualquier intento de reingreso que presente un token ya procesado dentro de la ventana de tiempo activa.

II-C2. 2. Control analítico y proactivo de la morosidad: A diferencia de los sistemas tradicionales que operan de manera pasiva y tardía, SocioUnido introduce un enfoque analítico y proactivo para la contención de la morosidad societaria. La plataforma integra un módulo especializado de inteligencia de negocios que procesa de manera continua la información de pagos y asistencias del club:

- **Monitoreo automatizado de comportamiento:** El sub-sistema analiza de manera regular el historial de pagos y el registro físico de accesos del socio. Esto permite identificar de forma automática patrones de inactividad o retrasos incipientes en el pago de la cuota social, alertando a la administración antes de que la deuda se consolide críticamente.
- **Tableros de control proactivos:** A través del panel web centralizado, las autoridades del club visualizan métricas clave de salud financiera y alertas parametrizadas. Esto permite segmentar al padrón de socios según su nivel de regularidad, facilitando la toma de decisiones basada en datos reales de uso y recaudación.
- **Campañas automáticas de regularización:** Una vez detectada una desviación en el comportamiento de pago o asistencia, el sistema permite disparar de forma proactiva

recordatorios y facilidades transaccionales a través de canales digitales integrados, reduciendo drásticamente la tasa de abandono de los socios mediante gestiones oportunas.

II-C3. 3. Interacción cognitiva sin fricción: asistente conversacional (BotIn): Para superar la tasa de rebote asociada a la descarga e instalación de aplicaciones nativas complejas en poblaciones deportivas heterogéneas, SocioUnido incorpora a su ecosistema un robot conversacional (*bot*) inteligente denominado BotIn, desarrollado sobre la API oficial de Telegram:

- **Procesamiento de lenguaje natural sin rigidez:** El microservicio conversacional utiliza arquitecturas avanzadas de orquestación cognitiva (como *LangChain*) para comprender el lenguaje cotidiano, coloquial e informal de los socios, superando los rígidos esquemas jerárquicos de botones de las opciones numéricas tradicionales.
- **Orquestación y cobro dinámico:** Cuando un socio interactúa con el bot (por ejemplo, enviando mensajes como “Hola, quiero pagar las cuotas que debo” o “¿qué canchas hay libres esta tarde?”), el motor reconoce la intención del usuario (*intent recognition*), extrae los parámetros contextuales, consulta en tiempo real el microservicio de gestión contable central y devuelve de manera inmediata un enlace dinámico de pago integrado con la pasarela transaccional de Mercado Pago, o bien confirma una reserva. Esto reduce la fricción operativa a cero y digitaliza transacciones complejas en segundos desde el canal de comunicación nativo del usuario.

II-C4. 4. Arquitectura adaptativa de marca blanca para la democratización del fútbol regional: El diseño arquitectónico definitivo de SocioUnido se rige por un principio de adaptabilidad estética y económica enfocado en dotar a las comisiones directivas de las herramientas que tradicionalmente pertenecieron a los clubes de élite:

- **Infraestructura elástica en la nube:** Al desplegar la plataforma bajo un modelo *SaaS* multiinquilino (*multi-tenant*) distribuido mediante contenedores y unificado por un portal de enlace de aplicaciones de alto rendimiento (*KrakenD API gateway*), se suprime por completo la necesidad de inversiones iniciales en hardware local o mantenimiento especializado por parte del club deportivo.
- **Marca blanca visualmente neutra y adaptativa:** Para posibilitar la comercialización masiva del *software* de forma transparente a múltiples instituciones deportivas clientes sin generar sesgos, rivalidades institucionales o fricciones cromáticas, SocioUnido adopta un diseño visual base estrictamente acromático (escala de grises neutros). Las interfaces se personalizan y adaptan dinámicamente mediante hojas de estilo dinámicas que inyectan los colores corporativos, escudos e identidad institucional de cada club una vez consolidada la suscripción, garantizando un posicionamiento corporativo óptimo para cada cliente.

III. METODOLOGÍA APLICADA

III-A. El marco de desarrollo incremental y adaptativo

La complejidad del ecosistema modular de *SocioUnido* exigió la adopción de una metodología de trabajo que garantizara un ciclo de vida de desarrollo de *software* altamente estructurado, auditable y capaz de adaptarse dinámicamente a los requerimientos técnicos identificados en el análisis del dominio deportivo argentino. En consonancia con lo establecido por el reglamento, el proyecto se rigió bajo un enfoque iterativo e incremental basado en el marco de trabajo ágil *Scrum*. Este paradigma metodológico permitió fragmentar un alcance tecnológico masivo en incrementos funcionales de valor integrados de forma continua, asegurando que la arquitectura del sistema se mantuviera estable y operable a lo largo de las diversas fases del proyecto.

III-B. Evolución y transiciones metodológicas a lo largo de las 37 semanas

El ciclo de vida del Trabajo Profesional se extendió a lo largo de un período neto de 37 semanas cronológicas. Sin embargo, la dinámica del trabajo en equipo y la naturaleza de las tareas académicas y técnicas obligaron a realizar transiciones metodológicas estratégicas, abandonando la linealidad del marco *Scrum* en aquellas fases donde su aplicación resultaba ineficiente o contraproducente para los objetivos específicos de ingeniería :

1. **Fase 1: Ideación, viabilidad y anteproyecto (semanas 1 a 12):** Durante este período inicial, el equipo no aplicó el marco de trabajo *Scrum* debido a que el esfuerzo se centró en la exploración del mercado deportivo, la aplicación de los marcos analíticos de las 5 C y las 4 P para la evaluación y descarte de las 13 ideas preliminares de negocio, y el diseño de la topología preliminar de servicios bajo el modelo de arquitectura C4. La dinámica de trabajo se rigió por jornadas asíncronas de investigación comercial, reuniones semanales de debate y la posterior redacción y entrega del documento de anteproyecto formal.
2. **Fase 2: Desarrollo del producto mínimo viable bajo Scrum (semanas 13 a 25):** Esta fase de máxima intensidad de desarrollo y codificación del *software* se gobernó de forma estricta por el marco ágil *Scrum* a lo largo de un ciclo cerrado de trece *sprints* de una semana de duración cada uno. La cadencia semanal fue elegida para acelerar los ciclos de retroalimentación interna, permitiendo que al cierre de cada iteración el equipo dispusiera de un incremento funcional del sistema completamente verificado y desplegado en los entornos de pruebas previas a producción (*staging*).
3. **Fase 3: Refinación, validación final y cierre (semanas 26 a 37):** Tras congelar el código del producto mínimo viable (*MVP*), el equipo abandonó de forma consciente el marco *Scrum* para adoptar un esquema de trabajo sumamente flexible basado en incidentes (*issue-driven*) y tableros visuales *Kanban*. Esta transición permitió

canalizar de manera eficiente las correcciones de errores (*bugs*) surgidas de las pruebas cualitativas de usuario, aplicar las recomendaciones estéticas, instrumentar la auditoría metodológica de calidad final y preparar el material de soporte para la defensa oral académica ante el jurado examinador.

III-C. Distribución definitiva de roles y especialización del equipo de ingeniería

Para blindar la ejecución del Trabajo Profesional, la estructura organizativa de *SocioUnido* se fundamentó en la asignación de responsabilidades principales y áreas de soberanía (*ownership*). Si bien los cuatro integrantes del equipo se involucraron activamente de forma transversal en todas las fases del ciclo de vida del *software*, la designación de un líder especializado por área garantizó que cada componente contara con un control de calidad y auditoría interna rigurosa antes de su fusión a la rama principal de código.

Al contrastar la ejecución final con lo planificado originalmente en la propuesta de anteproyecto, se produjo un cambio de roles entre dos de los integrantes del equipo, justificado por la especialización técnica y la optimización de los flujos de trabajo :

- **Ascencio, Felipe Santino:** Se desempeñó como responsable de control de calidad (*QA*), documentación técnica, desarrollo de negocio y preventa. Lideró la planificación y ejecución de las baterías de pruebas funcionales y de aceptación, la definición metodológica de los criterios de calidad del proceso, la redacción de los documentos técnicos y el diseño de los simuladores interactivos de retorno de inversión para la validación comercial.
- **Ghosn, Lautaro Gabriel:** Ejerció el rol de responsable de arquitectura, producto, bases de datos e integraciones del sistema. Tuvo a su cargo el diseño y mantenimiento de la topología distribuida de microservicios, el modelado relacional e incremental de las bases de datos en la nube, el diseño e instrumentación de las firmas y contratos de las interfaces de programación de aplicaciones (*APIs*) y la priorización del Product Backlog general.
- **Guerrero, Martín:** Asumió el rol de responsable de la capa lógica de servidor (*backend*), inteligencia artificial y relaciones institucionales sectoriales. Martín asumió estas responsabilidades para centralizar con éxito la implementación física de los microservicios en Python y FastAPI, la lógica de seguridad criptográfica del servidor y la orquestación asíncrona del asistente conversacional (*BotIn*) sobre la API de Telegram.
- **Zielonka, Axel:** Asumió las funciones de responsable de interfaces de usuario (*front-end*), experiencia/interfaz de usuario (*UX/UI*) e identidad de marca, ejerciendo adicionalmente el rol fundamental de *scrum master* durante la Fase 2 del desarrollo. Axel lideró de forma destacada el desarrollo integral del panel web de administración web, la maquetación responsiva de las aplicaciones web progresivas (*PWA*) para socios y empleados de control, el diseño de prototipos de alta fidelidad centrados en

usabilidad y el estudio de diseño de marca blanca adaptativo. Como *scrum master*, custodió el cumplimiento de las ceremonias ágiles y facilitó la remoción de bloqueos técnicos del equipo.

III-D. Garantías de calidad y completitud: Definition of Ready (DoR) y Definition of Done (DoD)

Para mitigar la introducción de deuda técnica y asegurar que los incrementos funcionales del ecosistema mantuvieran estándares de nivel productivo, la gestión del flujo de tareas se rigió de forma estricta por dos compuertas de validación fundamentales: la *Definition of Ready* (DoR) y la *Definition of Done* (DoD), las cuales fueron ajustadas para representar hitos de transición precisos dentro de un equipo con alta especialización técnica.

III-D1. Definition of Ready (DoR): Estado de desarrollo completado: A los fines metodológicos de este proyecto, el estado de listo (*Ready*) no hace referencia al momento previo al inicio de una tarea en el tablero, sino al hito formal en el cual el desarrollador asignado da por finalizada la construcción técnica de un componente en su entorno de aislamiento local. Para que una historia de usuario alcance la DoR y sea presentada para revisión, debe cumplir ineludiblemente con los siguientes tres requisitos técnicos :

1. **Implementación completa de la lógica:** El código fuente correspondiente ha sido escrito en su totalidad, satisfaciendo de manera estricta todos los requerimientos funcionales descritos en la ficha técnica del ticket original.
2. **Verificación local unitaria:** El desarrollador ha ejecutado y superado las pruebas unitarias locales pertinentes utilizando el marco de trabajo *Pytest*, garantizando la nula presencia de errores de sintaxis o fallos de ejecución evidentes en el microservicio modificado.
3. **Apertura de solicitud de integración (Pull Request - PR):** El código local es subido al repositorio de GitHub correspondiente y se formaliza un Pull Request dirigido a la rama de desarrollo, desencadenando la ejecución automática de las canalizaciones de integración continua (*CI/CD*).

III-D2. Definition of Done (DoD): Validación de completitud y calidad: La *Definition of Done* (Definición de Terminado) representa el estado final, cerrado e irreversible de una tarea o requerimiento en el flujo de trabajo. Un elemento de la lista de tareas pendientes (*backlog*) solo se declara formalmente como *Done* cuando ha superado exitosamente el escrutinio de calidad automatizado y el responsable de área temático correspondiente valida e instrumenta de forma explícita la fusión del Pull Request hacia la rama principal (*main*) del repositorio :

- **Validación de control de calidad:** A cargo de Felipe Santino Ascencio. Se encarga de comprobar que la funcionalidad responda a los criterios de aceptación lógicos pactados con los interesados, que se adjunte la documentación técnica correspondiente y que la cobertura de

pruebas unitarias y de integración se sitúe por encima del umbral de calidad requerido.

- **Validación de arquitectura, bases de datos e integraciones:** A cargo de Lautaro Gabriel Ghosn. Verifica que las modificaciones del código fuente respeten los principios de diseño de microservicios, el API Gateway KrakenD y el esquema transaccional relacional, asegurando la óptima interoperabilidad y la integridad referencial de los datos de SocioUnido.
- **Validación de front-end, Mobile y UX/UI:** A cargo de Axel Zielonka. Examina que las interfaces en la PWA y en el panel web administrativo cumplan de forma rigurosa con la paleta tipográfica y los lineamientos de diseño neutros y adaptativos para marca blanca detallados en el estudio estético corporativo.
- **Validación de backend, inteligencia artificial e institucional:** A cargo de Martín Guerrero. Aprueba la corrección matemática de la lógica de servicios asíncronos de FastAPI, la precisión analítica del motor de regularización y la correcta interpretación de intenciones conversacionales por parte de la API de Telegram en el bot conversacional.

Solo cuando se superan estas revisiones cruzadas y se obtiene la aprobación formal en el repositorio (*Approve Review*), el Pull Request es fusionado, considerándose la historia de usuario oficialmente como *Done*.

III-E. Gestión visual del alcance y control de errores mediante GitHub Projects

Para dar soporte metodológico a la ejecución distribuida del proyecto, el equipo prescindió del uso de herramientas tradicionales de terceros para centralizar de manera integral toda la gestión administrativa, de tareas y control de incidentes dentro del ecosistema de la organización de GitHub de *SocioUnido*. El seguimiento y administración del alcance y los errores se dividió en dos tableros visuales especializados creados bajo la plataforma *GitHub Projects* :

1. **Tablero de seguimiento de tareas:** Un tablero visual dinámico de tipo *Scrumban* que permitía priorizar los requerimientos funcionales refinados por el responsable de producto, estimar la complejidad en puntos de historia e instrumentar las ceremonias de planificación semanal de cada *sprint* durante la Fase 2 del desarrollo.
2. **Tablero de gestión y control de incidentes (Bug Tracking Board):** Un panel dedicado exclusivamente al reporte, categorización y resolución ágil de errores de código. Todas las inconsistencias detectadas en las fases de estabilización, las alertas automáticas de vulnerabilidad reportadas por el agente OpenClaw y las incidencias lógicas señaladas por SonarCloud eran transformadas inmediatamente en tarjetas (*issues*) de alta prioridad dentro de este tablero.

Para evitar la acumulación de deuda técnica y garantizar una distribución equitativa del esfuerzo de ingeniería, el equipo implementó un protocolo sistemático de resolución de incidentes: durante las ceremonias de sincronización semanales de

Scrum, el equipo revisaba conjuntamente el tablero de errores. Cada incidente era evaluado formalmente, clasificándose de acuerdo a su nivel de severidad y criticidad operativa, y era asignado de inmediato al responsable de área correspondiente. Este mecanismo impidió que las tareas de depuración quedaran postergadas, integrándolas de forma armónica en el flujo de trabajo diario y balanceando de forma continua la carga operativa de cada integrante.

III-F. Control de versiones y topología descentralizada de repositorios

Como plataforma para el desarrollo colaborativo y el control de versiones del código fuente de *SocioUnido*, se seleccionó GitHub. Su elección se fundamentó en la robustez de sus flujos de trabajo colaborativos y en el aprovechamiento de las licencias institucionales académicas provistas por la Facultad de Ingeniería de la Universidad de Buenos Aires.

Dada la arquitectura descentralizada basada en microservicios, el control del código fuente de SocioUnido se estructuró a través de un esquema multirepositorio alojado en la organización del equipo.

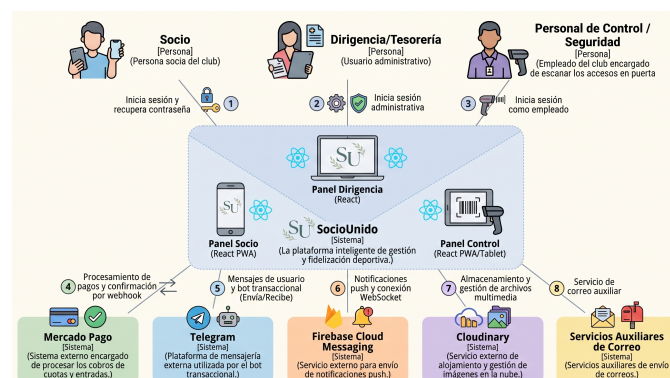


Figura 1. Diagrama de Contexto (C1) simplificado del ecosistema SocioUnido, ilustrando las fronteras de integración con actores humanos y servicios externos (Mercado Pago, Telegram, Cloudinary, Firebase).

III-G. Automatización de tareas mediante canalizaciones de integración y despliegue continuo (CI/CD)

La velocidad de entrega y la estabilidad del ecosistema se garantizaron mediante la automatización completa de los flujos de compilación, verificación y despliegue de *software* utilizando la herramienta de automatización integrada *GitHub Actions*. La adopción de esta tecnología eliminó la necesidad de configurar servidores de automatización externos complejos, ejecutando las tuberías directamente sobre la infraestructura del proveedor de manera integrada con cada Pull Request.

Las canalizaciones lógicas de integración continua (*CI*) implementaron una estrategia de barreras de calidad automáticas (*Quality Gates*) que impedían la fusión de código defectuoso a las ramas principales: al abrir un Pull Request, el flujo disparaba de forma automática las rutinas de formateo de código, verificaba que no existieran fallos de sintaxis o empaquetado, ejecutaba la batería completa de pruebas unitarias asíncronas

con *Pytest* y computaba el porcentaje de cobertura de código (*code coverage*).

Solo si el código fuente superaba con éxito el 100 % de las pruebas de aserción automatizadas y la cobertura de los tests se mantenía por encima del umbral fijado en el plan de calidad, la compuerta de integración habilitaba el botón de fusión para el responsable de área.

Una vez aprobada y fusionada la rama de desarrollo de forma asíncrona, se activaban automáticamente las canalizaciones de despliegue continuo (*CD*) para actualizar en tiempo real los entornos productivos elásticos en la nube :

- Las interfaces de usuario de *frontend* se compilaban y alojaban de manera directa sobre la plataforma de computación elástica *Vercel*, aprovechando su alta optimización para aplicaciones React.
- Los componentes de *backend* se empaquetaban en contenedores de infraestructura ligera mediante imágenes Docker y se desplegaban de manera ininterrumpida sobre la plataforma de servicios en la nube *Render*.

III-H. Inspección profunda y ciberseguridad continua: Dependabot y SonarCloud

Para asegurar que SocioUnido se mantuviera en el estado del arte de la ciberseguridad y la ingeniería de *software*, el ciclo de vida de desarrollo incorporó un enfoque proactivo de DevSecOps, integrando herramientas de análisis estático y de protección de cadena de suministro en cada repositorio de la organización :

- **Monitoreo continuo de dependencias (Dependabot):** Al estar basados en lenguajes como Python y Node.js, los microservicios dependen de una amplia red de paquetes y librerías de terceros susceptibles a brechas de seguridad y exposiciones públicas de vulnerabilidades (CVEs). Para neutralizar este riesgo de forma automatizada, se configuró Dependabot de manera transversal en todos los repositorios. La herramienta escanea ininterrumpidamente los manifiestos de dependencias, cruza los datos con bases de datos globales de vulnerabilidades actualizadas en tiempo real y, al detectar una brecha, genera de manera autónoma un Pull Request con la actualización exacta a la versión segura mínima, garantizando la paridad con la suite de pruebas automatizadas del pipeline de CI antes de su fusión.
- **Análisis estático de código estricto (SonarCloud):** Mientras que Dependabot auditaba el código de terceros, la calidad de la lógica de negocio propia desarrollada por los estudiantes se sometió al análisis estático de caja blanca mediante la integración directa de SonarCloud como motor de SAST (Static Application Security Testing). En cada ejecución, SonarCloud analizaba la complejidad ciclomática del código, identificaba puntos propicios a fallos de desbordamiento, detectaba la presencia de código muerto o duplicaciones, y auditaba la cobertura de código de los Pull Requests. Los badges de control del estado de SonarCloud de cada microservicio se expusieron de manera dinámica en el repositorio central `.github`,

conformando una torre de control transparente y auditable que garantizaba que ninguna pieza de código fuera integrada con deuda técnica acumulada.

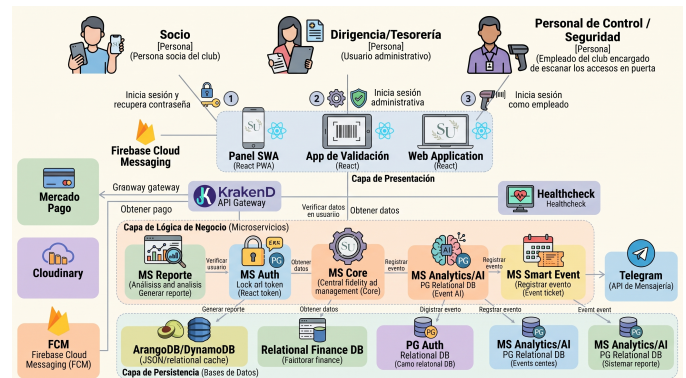


Figura 2. Diagrama de Contenedores (C2), detallando el aislamiento de las interfaces de *frontend* respecto a la capa de lógica de negocio (Microservicios en Python/FastAPI) y la capa de persistencia.

IV. EXPERIMENTACIÓN, VALIDACIÓN Y CONTROL DE CALIDAD

Para garantizar la robustez, seguridad y fiabilidad de la plataforma **SocioUnido**, se ejecutó una estrategia integral de pruebas a lo largo de todo el ciclo de desarrollo. Este plan de validación certificó que el sistema cumple con la totalidad de los requerimientos funcionales y no funcionales especificados en las etapas tempranas del proyecto.

IV-A. Roles, responsabilidades y grado de automatización

La especificación, diseño y supervisión del plan de pruebas estuvo a cargo del **Responsable de Control de Calidad (QA)**. Su rol consistió en definir los casos de prueba, configurar los entornos de automatización y ejecutar las pruebas manuales exploratorias. En concordancia con las prácticas ágiles, los desarrolladores de cada módulo (*front-end*, *back-end*, *IA*) fueron responsables de escribir y mantener las pruebas unitarias de su propia implementación.

La estrategia adoptó un enfoque de “Pirámide de Pruebas”, priorizando un alto grado de automatización en las capas inferiores para agilizar la Integración y Despliegue Continuo (CI/CD), reservando las pruebas manuales para la validación de usabilidad y flujos complejos:

- **Pruebas unitarias (automatizadas):** Se validó el correcto funcionamiento de funciones y métodos aislados utilizando *frameworks* estándar de la industria, integrados directamente en los repositorios.
- **Pruebas de integración (automatizadas):** Se verificó la correcta comunicación entre los microservicios, la base de datos relacional y las API de terceros (pasarelas de pago, Resend y Telegram).
- **Pruebas de extremo a extremo [End-to-End] (semiautomatizadas):** Se simulaban flujos completos de usuario (por ejemplo, el pago de una cuota desde la PWA del socio hasta su reflejo en el panel de control administrativo).

IV-B. Validación funcional y no funcional

El conjunto de pruebas funcionales se centró en auditar los módulos principales (*core*) del ecosistema:

- **Gestión administrativa:** Verificación de los flujos de creación, lectura, actualización y borrado (CRUD) del padrón de socios, y la correcta renderización de métricas.
- **Procesamiento de pagos:** Validación de la conciliación de pagos y webhooks con pasarelas externas.
- **Asistente conversacional (PLN):** Pruebas de caja negra sobre el bot de Telegram (“BotIn”), inyectando frases de intención variada (texto formal, informal, modismos) para validar la precisión del reconocimiento de lenguaje natural.
- **Acceso inteligente [Smart Access] (TOTP):** Comprobación rigurosa del algoritmo criptográfico y evaluación de la correctitud de su funcionamiento offline.

A nivel no funcional, la plataforma fue sometida a pruebas de rendimiento simulando picos extremos de tráfico para asegurar la estabilidad durante los días de partido. Asimismo, se verificó la compatibilidad multiplataforma de las interfaces web y móviles, y se auditó el cifrado de datos en reposo y en tránsito.

IV-C. Validación con usuarios finales y UX/UI

Considerando que la usabilidad y la adecuación al mercado son pilares vitales para la adopción masiva del sistema, se ejecutó una etapa de validación participativa orientada a medir la satisfacción real de los usuarios. Que al hincharle le resulte intuitiva la aplicación y al trabajador del club le aporte soluciones prácticas son factores críticos de éxito.

- **Validación de producto con clubes:** Se llevaron a cabo reuniones con personal administrativo y directivo de diferentes instituciones para presentarles el producto y exponer sus valores agregados. El objetivo central de estos encuentros fue obtener *feedback* directo sobre la utilidad real de la solución, relevar puntos de mejora y analizar las principales diferencias competitivas de SocioUnido frente a las plataformas de gestión preexistentes en el mercado.
- **Validación masiva mediante Google Forms:** Se distribuyeron encuestas estructuradas a una muestra significativa de socios finales (*beta-testers*) luego de que interactuaran con la aplicación móvil (PWA). Este formulario recolectó métricas precisas sobre la intuición del diseño, la usabilidad y la fluidez de los procesos dentro de la aplicación. El *feedback* obtenido fue un insumo crítico que permitió refinar la interfaz y garantizar una experiencia de usuario (UX/UI) de altísima calidad antes del despliegue final.

IV-D. QA automatizado con agente de OpenClaw

En el marco del desarrollo de SocioUnido, alcanzar la más alta calidad en los estándares de ciberseguridad y de interoperabilidad se planteó como un objetivo innegociable. Para que el ecosistema se mantenga a la vanguardia, resulta

clave que la arquitectura y los *endpoints* estén preparados no solo para usuarios humanos, sino también para integrarse de manera fluida y segura con agentes autónomos de terceros e Inteligencias Artificiales.

Para llevar a cabo una validación rigurosa, se implementó **OpenClaw**, un agente de IA diseñado específicamente para el *testing* automatizado y el QA integral de repositorios. Su adopción se justifica por su capacidad para comprender el contexto del código e identificar vulnerabilidades lógicas complejas.

IV-D1. Directrices del agente: La implementación técnica del agente se basó en un marco de instrucciones precisas (*prompting*) que auditan estrictamente tres ejes fundamentales:

- **Seguridad de la implementación:** Búsqueda exhaustiva de vulnerabilidades, haciendo foco en los estándares de la industria (OWASP Top 10) y prevención de vectores de ataque.
- **Prevención de fuga de datos:** Asegurar rigurosamente que el código fuente no exponga credenciales, tokens de acceso ni información privada de los usuarios.
- **Preparación para IA y automatización:** Evaluar si la estructura de los datos está diseñada para garantizar una interacción sin fricciones con futuras automatizaciones.

IV-D2. Resultados obtenidos: A continuación, se detalla el análisis exhaustivo de seguridad realizado por OpenClaw sobre los repositorios principales que conforman la arquitectura de SocioUnido.

Aplicación móvil PWA (Socios)

- **Fuga de información sensible:** Se identificó que, ante fallos inesperados, el componente capturador de errores exponía la traza completa del sistema (*stack trace*) en pantalla.
- **Manipulación de rutas API:** Se detectó una concatenación directa e insegura de parámetros en peticiones HTTP, dejando abierta la posibilidad de alterar rutas (*Path Traversal*).
- **Gestión de encabezados de autorización:** El módulo de peticiones enviaba encabezados con valores nulos al navegar de forma anónima.
- **Exposición de datos y tokens:** El contexto global registraba en consola información personal y tokens de notificaciones.
- **Metadatos de idioma en documentos que confunden a agentes de IA.**

Aplicación móvil PWA (Trabajadores)

- **Fuga de información sensible:** Exposición de la traza completa del sistema en pantalla ante fallos.
- **Gestión de encabezados:** Envío de encabezados nulos en navegación anónima.
- **Políticas de seguridad inexistentes:** Carencia de políticas de restricción de contenido.

API Gateway

- **Elevación de privilegios:** Módulos sensibles carecían de verificación de permisos.

- **Ausencia de límites de tráfico:** Rutas sin restricción de peticiones por minuto.
- **Privilegios máximos en contenedores:** Ejecución del servicio bajo el usuario `root`.
- **Desactivación de validación de llaves:** Descarga insegura de llaves criptográficas.

Microservicio de accesos

- **Almacenamiento en texto plano:** Las semillas secretas para los códigos dinámicos (TOTP) se almacenaban sin cifrado.
- **Fuga de tokens en registros.**
- **Valores predeterminados rígidos.**
- **Políticas de origen cruzado ausentes.**

Microservicio de analíticas

- **Procesamiento deficiente de mensajes:** Lectura asíncrona sin validación de estructura.
- **Autenticación vulnerable (Timing Attack):** Verificación de tokens mediante comparaciones de texto convencionales.
- **Contratos de interfaz incompletos.**

Microservicio de autenticación

- **Políticas de contraseñas laxas:** Permisividad de claves demasiado cortas en el registro.
- **Estructuras genéricas:** Uso de diccionarios sin tipo, impidiendo esquemas claros para IA.

Microservicio de bot conversacional (PLN)

- **Falta de validación de firmas:** Ausencia de verificación criptográfica de las peticiones del proveedor (Telegram/Meta).
- **Exposición de datos en servidor:** Registros en consola de mensajes.
- **Caídas por datos invalidos:** Excepciones críticas ante mensajes con formatos inesperados.

Microservicio de gestión del club (Core)

- **Exposición de arquitectura interna:** Devolución de errores crudos revelando la estructura de las tablas de la base de datos.
- **Inconsistencia en contratos de datos:** Múltiples salidas de información con fallas de validación de esquemas.

Microservicio de pagos

- **Omisión de autenticación en cobros:** Punto de acceso principal sin validación de clave interna.
- **Ausencia de verificación en webhooks:** Falta de validación de la firma criptográfica.
- **Inseguridad en intermediario de eventos:** Conexión a Redis con validación SSL/TLS.
- **Gestión de credenciales:** Claves secretas débiles y expuestas en archivos de migración.

Panel administrativo web

- **Exposición de credenciales.**
- **Sobrecarga de memoria:** Descarga del padrón completo de socios sin paginación.
- **Vulnerabilidad XSS:** Inyección de enlaces e imágenes sin sanitización previa.

- **Manipulación de rutas (Path Traversal):** Identificadores dinámicos no codificados.

IV-D3. Acciones tomadas y refactorización: Las observaciones del agente automatizado impulsaron una refactorización integral en todos los módulos de la aplicación:

- **Cifrado y credenciales:** Se eliminaron contraseñas por defecto y claves en el código, cifrando las semillas TOTP con algoritmos simétricos.
- **Auditoría de dependencias:** Se exigieron librerías certificadas para mitigar ataques a la cadena de suministro.
- **Sanitización y paginación:** Se aplicaron políticas CSP, codificación de parámetros y paginación del lado del servidor, previniendo fugas masivas de datos.
- **Concurrencia y privilegios:** Se integraron bloqueos de hilos en servicios compartidos y se aislaron los contenedores ejecutándolos sin privilegios administrativos (`root`).
- **Protección criptográfica:** Se adoptaron comparaciones de tiempo constante (mitigando *Timing Attacks*) y se forzó la validación de firmas en *webhooks*.
- **Contratos para IA:** Se tiparon estrictamente los esquemas, estandarizando las respuestas de error para no revelar la arquitectura interna.

En conclusión, las exhaustivas auditorías de OpenClaw y sus posteriores correcciones dotaron a SocioUnido de una infraestructura robusta, cibersegura y preparada para integrarse sin fricciones con futuras plataformas y agentes inteligentes.

V. CONCLUSIONES

Las conclusiones de este documento recapitulan los aspectos centrales descritos en las secciones anteriores, evaluando el alcance funcional del Producto Mínimo Viable (MVP) entregado, el ecosistema documental construido y el impacto real de la plataforma. Esta visión holística permite dimensionar el verdadero profesionalismo del proyecto y cómo este materializa la formación integral recibida a lo largo de la carrera.

V-A. Resultados principales: Ecosistema organizativo y repositorios

Absolutamente toda la implementación técnica, lógica y documental del Trabajo Profesional reside de forma centralizada en nuestra organización oficial de GitHub. La magnitud del proyecto exigió una arquitectura sumamente distribuida, componiéndose de **27 repositorios y 2 proyectos Kanban**.

V-A1. Gestión ágil y control de incidencias (Proyectos Kanban):

- **Centro de monitoreo de tareas:** Tablero estructurado bajo la metodología Kanban que rigió la ejecución de los 13 *sprints* de desarrollo. Permitted gestionar el *backlog* de manera dinámica, asignando responsables, pesos (esfuerzo) y prioridades a cada historia de usuario.
- **Centro de monitoreo de bugs:** Espacio homólogo dedicado exclusivamente a la detección, seguimiento y resolución de errores (*bugs*) y deudas técnicas reportadas por los auditores automáticos (SonarCloud/OpenClaw).

V-A2. *Repositorios privados (Lógica de negocio e infraestructura)*: Con el fin de resguardar la propiedad intelectual, la seguridad criptográfica y los derechos de autor de las implementaciones subyacentes, los repositorios *back-end* se mantienen en carácter privado. Sin embargo, **ante cualquier requerimiento académico, de evaluación o interés técnico justificado, el equipo está totalmente a disposición para otorgar los accesos correspondientes**. El código fuente de estos repositorios está documentado internamente siguiendo los más altos estándares y buenas prácticas (*self-documenting code*).

V-A3. *Repositorios públicos (Interfaces y portales documentales)*: La totalidad de las interfaces cliente y la documentación asociada a los repositorios privados se mantienen en carácter público para demostrar la transparencia, alcance y arquitectura del proyecto sin comprometer credenciales sensibles.

Portales documentales de la arquitectura (Back-end): Para cada microservicio privado existe un repositorio espejo terminado en `-doc`. Estos alojan portales generados con JustTheDocs que explican exhaustivamente la arquitectura del servicio original: diagramas de contexto, justificación tecnológica, métricas de implementación y el listado de *endpoints* públicos.

Interfaces gráficas (Front-end): Exponen el código y sus propias páginas documentales con vistas principales del sistema (plataforma-web, aplicacion, control-de-accesos).

V-B. *El inmenso valor documental del proyecto*

El hito más destacable de SocioUnido, a la par de su alcance tecnológico, es su aporte documental. Se han construido **16 páginas plenamente documentales con JustTheDocs** que abarcan el Trabajo Profesional de punta a punta (8 de arquitectura, 3 visuales y 5 de gestión global). Particularmente, el repositorio *documentacion* aloja una vasta cantidad de material complementario:

- El Anteproyecto y las versiones iterativas de la Entrega Intermedia.
- Evidencia de validación del producto (entrevistas a trabajadores y encuestas de usabilidad).
- El Informe Final definitivo y diagramas del Modelo C4 / DER.
- Redacciones de discursos comerciales (*Pitch*) y desarrollo de identidad visual de marca.
- Más de **25 anexos formales** analizando aristas técnicas, legales y de gestión.
- Decenas de videotutoriales explicativos de la plataforma con sus manuales de uso.

V-C. *Identidad comercial, innovación y contribución a otras entidades*

Desde su concepción, SocioUnido fue tratado como un producto real. Para respaldar esta visión, se consolidó una **Página publicitaria (Landing Page)** y un **Canal de YouTube**

para validación y tutoriales. Este enfoque comercial fue validado de forma empírica y constante mediante reuniones con trabajadores de clubes reales y encuestas a usuarios finales.

A partir de este desarrollo, el Trabajo Profesional ha generado **innovaciones tecnológicas** significativas que contribuyen directamente a la modernización, inclusión y saneamiento de las instituciones deportivas (especialmente en las categorías de ascenso):

1. **Innovación criptográfica (Infraestructura offline)**: Adaptación del algoritmo TOTP para la validación de accesos en estadios sin conexión a internet, mitigando el colapso de las redes 4G/5G en eventos masivos y erradicando el fraude sin requerir hardware costoso.
2. **Innovación predictiva**: Desarrollo de herramientas para predecir el riesgo de fuga societaria basándose en históricos y métricas definidas por el club, transformando la cobranza reactiva en un modelo de retención proactiva.
3. **Innovación en accesibilidad (PLN)**: Democratización del sistema mediante un asistente omnicanal en Telegram con Procesamiento de Lenguaje Natural, eliminando la fricción tecnológica para usuarios de todas las edades.

V-D. *Impacto en la formación: Conocimientos aplicados*

El diseño, arquitectura y desarrollo de la plataforma SocioUnido no es un esfuerzo aislado, sino la síntesis y aplicación directa de los conocimientos adquiridos a lo largo de toda la carrera de Ingeniería en Informática. La resolución de las problemáticas de las instituciones deportivas exigió un enfoque multidisciplinario:

- **Fundamentos matemáticos, ciencias básicas y algoritmos**: *Física* proveyó el conocimiento sobre la propagación de señales, justificando la validación de ingresos de forma *offline* en estadios repletos. *Álgebra A* y *Álgebra Lineal* fueron indispensables para comprender la criptografía (aritmética modular y *hash*) detrás de la innovación TOTP. *Química* aportó la perspectiva de las limitaciones de las baterías de los dispositivos de escaneo móvil a la intemperie. *Algoritmos y Estructuras de Datos*, y *Teoría de Algoritmos* constituyeron el núcleo de eficiencia, optimizando las búsquedas y el filtrado de enormes padrones. Finalmente, *Análisis Matemático* y *Probabilidad y Estadística* aportaron el marco teórico para la evaluación y limpieza de datos.
- **Ingeniería de software, arquitectura y calidad**: *Ingeniería de Software I* y *II* guiaron la concepción del ciclo de vida, la aplicación de Scrum, arquitecturas de microservicios (alta cohesión y bajo acoplamiento) y el *Delivery Pipeline* (CI/CD). Materias como *Taller de Programación* y *Paradigmas de Programación* forjaron las bases de pruebas automatizadas (TDD) y el desarrollo de código limpio y escalable.
- **Sistemas, redes y bases de datos**: *Organización del Computador* y *Sistemas Operativos* permitieron comprender la arquitectura a bajo nivel y el manejo de memoria en contenedores. *Base de Datos* aseguró la

persistencia y normalización mediante transacciones concurrentes en PostgreSQL. *Programación Concurrente, Redes y Sistemas Distribuidos I* resultaron la columna vertebral de la plataforma Cloud, garantizando balanceo de carga, protocolos de red (HTTP/TCP), APIs RESTful y tolerancia a fallos.

- **Ciencia de datos y Ciberseguridad:** Los conocimientos de *Ciencia de Datos* sirvieron para procesar grandes volúmenes de transacciones y extraer características proyectando soluciones de retención. En paralelo, el *Taller de Seguridad Informática* aportó las técnicas de defensa en profundidad, sanitización de datos y gestión de roles (JWT) para proteger la información.
- **Emprendedurismo, gestión y contexto profesional:** *Introducción al Conocimiento de la Sociedad y el Estado e Introducción al Pensamiento Científico* brindaron el pensamiento crítico para entender la dinámica socio-política de los clubes. *Empresas de Base Tecnológica I y II* sentaron las bases corporativas, financieras y legales del modelo SaaS B2B, estrategias de *pricing* y propiedad intelectual. Por último, *Gestión del Desarrollo de Sistemas Informáticos* aportó la estructura ejecutiva para estimar tiempos, liderar al equipo y gestionar los riesgos del proyecto.

V-E. Cierre y reflexión final

En síntesis, SocioUnido no es un mero ensamblaje de tecnologías, sino el producto maduro de un ingeniero informático formado integralmente en la Universidad de Buenos Aires (UBA), capaz de integrar ciencias básicas, aplicadas, y competencias de gestión para resolver problemas complejos de la sociedad y la industria.

El código fuente en su totalidad cuenta con más de **1.200 commits** registrados y **290** revisiones cruzadas (*Pull Requests*) evaluadas. El volumen estructural desarrollado impactó en aproximadamente **400.000** líneas de código a lo largo de este año de desarrollo continuo.

Por esto, y por el impacto real del producto construido, el equipo no podría estar más orgulloso de los resultados obtenidos. Tenemos la profunda convicción de que el Trabajo Profesional que estamos entregando representa el más alto estándar de excelencia institucional. Haber fortalecido el proyecto en todos los aspectos evaluables (tecnológicos, comerciales, documentales y arquitectónicos) demuestra que el trayecto formativo de nuestra carrera ha rendido sus frutos, cerrando esta etapa universitaria con un broche de oro materializado en este impresionante entregable final.

VI. REFERENCIAS

REFERENCIAS

- [1] Anthropic. (s.f.). *Claude Documentation*. Recuperado de <https://docs.anthropic.com/>
- [2] Cloudinary. (s.f.). *Cloudinary Documentation*. Recuperado de <https://cloudinary.com/documentation>

- [3] Consejo Directivo, Facultad de Ingeniería, Universidad de Buenos Aires. (22 de diciembre de 2025). *Resolución N° RESCD-2025-1124-E-UBA-DCT_FI: Reglamento de Trabajo Integrador Final para carreras de grado FIUBA*. Recuperado de https://cms.fi.uba.ar/uploads/RESCD_2025_1124_Reglamento_de_Trabajo_Integrador_Final_para_carreras_de_grado_6da340c642.pdf
- [4] Docker Inc. (s.f.). *Docker Documentation*. Recuperado de <https://docs.docker.com/>
- [5] Figma. (s.f.). *Figma Help Center*. Recuperado de <https://help.figma.com/>
- [6] GitHub. (s.f.). *GitHub Docs (incluye GitHub Actions y Dependabot)*. Recuperado de <https://docs.github.com/>
- [7] Global.fan. (s.f.). *Global.fan - Plataforma de gestión deportiva y engagement*. Recuperado de <https://global.fan>
- [8] Google. (s.f.). *Firebase Documentation*. Recuperado de <https://firebase.google.com/docs>
- [9] Google. (s.f.). *Gemini AI*. Recuperado de <https://deepmind.google/technologies/gemini/>
- [10] KrakenD. (s.f.). *KrakenD API Gateway Documentation*. Recuperado de <https://www.krakend.io/docs/>
- [11] Mercado Pago. (s.f.). *Mercado Pago Developers*. Recuperado de <https://www.mercadopago.com.ar/developers/es/docs>
- [12] Meta Platforms, Inc. (s.f.). *React Documentation*. Recuperado de <https://react.dev/>
- [13] Neon. (s.f.). *Neon Serverless Postgres Documentation*. Recuperado de <https://neon.tech/docs/>
- [14] OpenAI. (s.f.). *OpenAI Documentation*. Recuperado de <https://platform.openai.com/docs/>
- [15] Python Software Foundation. (s.f.). *Python 3 Documentation*. Recuperado de <https://docs.python.org/3/>
- [16] Redis. (s.f.). *Redis Documentation*. Recuperado de <https://redis.io/docs/>
- [17] Render. (s.f.). *Render Documentation*. Recuperado de <https://render.com/docs>
- [18] Resend. (s.f.). *Resend Documentation*. Recuperado de <https://resend.com/docs>
- [19] SonarSource. (s.f.). *SonarCloud Documentation*. Recuperado de <https://docs.sonarsource.com/sonarcloud/>
- [20] SQLAlchemy. (s.f.). *SQLAlchemy Documentation*. Recuperado de <https://docs.sqlalchemy.org/>
- [21] Telegram. (s.f.). *Telegram Bot API*. Recuperado de <https://core.telegram.org/bots/api>
- [22] Tiangolo. (s.f.). *FastAPI Documentation*. Recuperado de <https://fastapi.tiangolo.com/>
- [23] Todo Noticias (TN). (28 de enero de 2026). *La AFA reveló cuántos socios tienen los clubes de Primera División: cómo quedó el ranking*. Recuperado de <https://tn.com.ar/deportes/futbol/2026/01/28/la-afa-revelo-cuantos-socios-tienen-los-clubes-de-primera-division-como-quedo-el-ranking>
- [24] Vercel. (s.f.). *Vercel Documentation*. Recuperado de <https://vercel.com/docs>
- [25] Vite. (s.f.). *Vite Documentation*. Recuperado de <https://vitejs.dev/>